

WEB APPLICATION VULNERABILITY ASSESSMENT & PENETRATION TESTING REPORT

CommerceHub Merchant Portal

Prepared for Nexora Retail Services Pvt. Ltd.

Classification: Confidential

Report ID: COS-NEX-WEB-2026-062

Report Date: 05-Jun-2026



NEXORA

Retail Services Pvt. Ltd.

Prepared by

CipherOak Security LLP

Application Security Assessment Team

Document Control & Distribution

Client	Nexora Retail Services Pvt. Ltd.
Assessed Application	CommerceHub Merchant Portal and supporting API services
Assessment Type	Grey-box Web Application Vulnerability Assessment and Penetration Testing
Environment	Staging environment with production-equivalent configuration and test data
Testing Window	20-May-2026 to 27-May-2026
Report Date	05-Jun-2026
Report Classification	Confidential - restricted to Nexora Retail Services and authorized service providers
Report ID	COS-NEX-WEB-2026-062

Version	Date	Author / Reviewer	Change Summary
1.0	31-May-2026	Rohan Shah, Lead Consultant	Initial technical observations consolidated
1.1	03-Jun-2026	Meera Iyer, QA Reviewer	Evidence review and CVSS validation
1.2	05-Jun-2026	CipherOak Security LLP	Final management-ready report issued

Distribution note: This document contains sensitive security information. Evidence snippets are redacted where needed and should not be shared outside approved distribution channels.

Report Navigator

1	Executive Summary and Risk Dashboard
2	Introduction, Objectives, Assumptions and Limitations
3	Engagement Scope, Testing Credentials and Exclusions
4	Sampling Criteria, Audit Team and Timeline
5	Audit Methodology, Criteria and Tools Used
6	Summary of Detailed Observations
7	Detailed Vulnerability Observations and Proofs of Concept
8	Remediation Roadmap and References

The report layout uses dashboard-style summaries for management readers and observation cards for remediation teams. Each finding contains preconditions, reproducible evidence, business impact, root cause, remediation guidance and validation checks.

1. Executive Summary

CipherOak Security LLP performed a grey-box VAPT of Nexora Retail Services' CommerceHub Merchant Portal, covering the staging web application, API layer and related file delivery paths. The assessment identified 8 validated vulnerabilities, including 2 Critical and 5 High severity issues that should be remediated before the next production promotion.

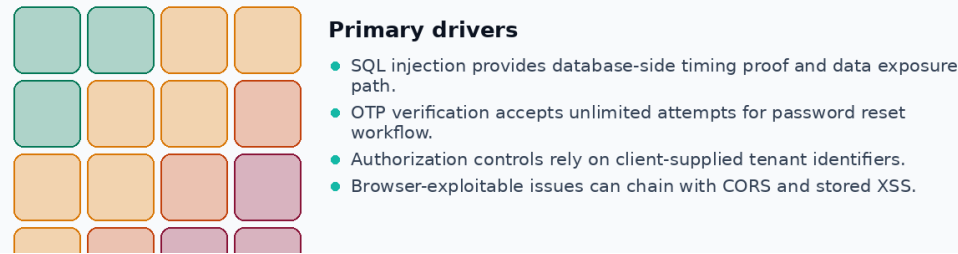
The highest-risk findings are a time-based SQL injection in the order search API and unrestricted OTP attempts in the password reset workflow. Both issues present realistic paths to unauthorized data access or account takeover in the tested environment. Several browser-side and configuration weaknesses can also be chained with authenticated sessions, especially the CORS origin reflection and stored cross-site scripting findings.

Executive Risk Dashboard

8 validated findings | 2 Critical | 5 High | 1 Medium | 0 Low



Likelihood x Impact Heatmap



Management conclusion: CommerceHub demonstrates functional security controls in several areas, including TLS enforcement at the edge and role-based UI restrictions. However, server-side authorization, input handling and account recovery controls require immediate hardening. Remediation should prioritize Critical findings within 7 calendar days and High findings within 15 calendar days.

2. Introduction, Objective, Limitations and Assumptions

Introduction

CommerceHub Merchant Portal is a web-based portal used by merchant partners to manage orders, invoices, refunds, profile settings, documents and support requests. The assessment focused on validating whether the application and supporting APIs resist common web application attack paths under authenticated and unauthenticated test conditions.

Objective

The objective was to identify exploitable vulnerabilities, validate business impact using controlled proof-of-concept activity, map findings to recognized web security standards, and provide remediation guidance that can be verified during retesting.

Limitations / Assumptions

Assumption / Constraint	Description
Time-boxed testing	The assessment was conducted within an agreed testing window; results represent security posture during that period.
Staging equivalence	Nexora confirmed that staging uses production-equivalent code paths, middleware and security controls, with test data.
Data handling	Evidence was captured using dedicated test tenants and redacted where sensitive identifiers or secrets appeared.
Control validation	Findings were validated manually; automated scanner output was not accepted as evidence without manual confirmation.
No service disruption	Denial-of-service, destructive database writes and resource-exhaustion techniques were not performed.

3. Engagement Scope, Testing Credentials and Deviations

Engagement Scope

Asset	URL / Identifier	In-Scope Functions
Web Portal	https://merchant-stage.nexora-retail.example	Merchant dashboard, order search, invoices, refunds, support ticketing, document upload
API Gateway	https://api-stage.nexora-retail.example	REST APIs used by portal, mobile browser flows and support console
Asset Delivery	https://assets-stage.nexora-retail.example	Uploaded merchant documents and static content delivery
Identity Provider	https://auth-stage.nexora-retail.example	Login, MFA, password reset and token issuance flows

Testing Credentials

Role	Username	Authentication	Purpose
Merchant Admin	qa.merchant.admin@nexora.local	MFA enabled	Orders, refunds, document upload, profile management
Merchant Operator	qa.merchant.operator@nexora.local	MFA enabled	Orders, invoices and support tickets
Vendor Viewer	qa.vendor.viewer@nexora.local	MFA enabled	Read-only merchant records for tenant NTX-MER-021
Support Agent	qa.support.agent@nexora.local	MFA enabled	Back-office support ticket review

Credential handling: Passwords, recovery seeds and MFA backup codes were provided through Nexora's approved secure channel and are not stored in this report.

Exclusions / Additions / Deviations

Category	Details
Excluded	Production environment, payment gateway settlement rails, third-party logistics APIs, mobile native applications and social engineering.
Added	CORS behavior, asset domain content handling and password reset OTP behavior were added after initial reconnaissance indicated elevated risk.
Deviation	Automated request-rate testing was capped to agreed thresholds and executed only against designated test accounts.

4. Sampling Criteria, Audit Team and Timelines

Sampling Criteria

Test cases were selected using a risk-weighted sampling model. Priority was given to functions processing personal data, financial instructions, tenant-separated data, role transitions, file ingestion, state-changing actions, account recovery and unauthenticated endpoints exposed at the application edge.

Sample Area	Selection Rationale
Authorization	Tenant ID, invoice ID, refund ID, support ticket ID and admin function access checks across four roles.
Input handling	Search, filters, ticket comments, upload metadata, JSON bodies and query parameters.
Authentication	Login, MFA, password reset, session termination and token validation paths.
Data exposure	Profile, invoice, order, debug, document and support APIs.
Browser security	CORS, CSP, active content handling, cookie attributes and cross-origin behavior.

Details of Auditing Team

Name	Role	Responsibilities
Rohan Shah	Lead Security Consultant	Engagement lead, authorization testing, reporting
Meera Iyer	Senior Application Security Analyst	API testing, injection validation, retest criteria
Nikhil Rao	Security Engineer	Authentication, session management and tooling
Priya S. Menon	Quality Reviewer	Evidence review, CVSS validation and report quality control

Audit Activities and Timelines

Date / Window	Activity	Outcome
20-May-2026	Kick-off and access validation	Scope, credentials and rules of engagement confirmed
21-May-2026 to 22-May-2026	Reconnaissance and mapping	Application flows, API inventory, role matrix and test data mapped
23-May-2026 to 26-May-2026	Manual exploitation and validation	Business logic, auth, injection, browser security and upload controls tested
27-May-2026	Evidence review	Screenshots, HTTP traces and technical impact confirmed
31-May-2026 to 05-Jun-2026	Reporting	Risk rating, remediation plan and management summary finalized

5. Audit Methodology, Criteria and Tools Used

Audit Methodology and Criteria / Standards Referred

Reference	How it was applied
OWASP Top 10 2025	Used for web application risk categorization and management-level mapping.
OWASP Web Security Testing Guide	Used as the primary manual testing guide for authentication, authorization, input validation, session management and configuration checks.
OWASP ASVS 5.0.0	Used as the control reference for application security verification and remediation expectations.
FIRST CVSS v3.1	Used for technical severity scoring. Each finding includes a CVSS vector string.
Nexora Rules of Engagement	Used to define test accounts, rate limits, prohibited actions and communication process.

Approach and Methodology

Phase	Activities
1. Planning	Scope validation, rules of engagement, credential onboarding, stakeholder contacts and evidence handling process.
2. Discovery	Crawl portal, proxy traffic, enumerate API routes, identify role-specific workflows and capture baseline behavior.
3. Vulnerability Identification	Manual testing supported by targeted scanners, payload variation, authorization matrix testing and workflow abuse testing.
4. Exploit Validation	Controlled PoC execution using test data only, with response evidence, screenshots and business impact analysis.
5. Risk Rating	CVSS v3.1 base score combined with business impact, data sensitivity and exploit preconditions.
6. Reporting	Root cause, remediation guidance, retest criteria, target owners and prioritization.

Type of Assessment and Testing Environment

Assessment Type	Grey-box web application VAPT with authenticated role-based testing and limited unauthenticated testing.
Testing Environment	Staging environment, production-equivalent application build, production-equivalent infrastructure policies and synthetic test data.
Connectivity	Auditor IP ranges allowlisted by Nexora for staging access. Browser and proxy traffic routed through CipherOak test workstation.
Evidence Handling	Screenshots, HTTP requests and response snippets sanitized for secrets, personal data and tenant identifiers.

Tools / Software Used

Tool	Purpose
Burp Suite Professional	Proxying, manual request replay, Intruder rate-limit validation and evidence capture
OWASP ZAP	Supplementary passive scanning and header review

Tool	Purpose
Postman / curl	API request reproduction and authorization matrix checks
Browser DevTools	DOM inspection, storage review, CORS verification and console validation
Nuclei and custom scripts	Targeted template checks and repeatability validation
JWT tooling	Token header/payload inspection and validation checks

6. Summary of Detailed Observations

ID	Observation	Severity	CVSS	OWASP Mapping	Status
F-01	Broken Object Level Authorization in Invoice Retrieval	High	8.1	A01: Broken Access Control	Open
F-02	Time-Based SQL Injection in Orders Search API	Critical	9.9	A05: Injection	Open
F-03	Password Reset OTP Endpoint Missing Rate Limiting	Critical	9.1	A07: Authentication Failures	Open
F-04	Stored Cross-Site Scripting in Support Ticket Comments	High	8.7	A05: Injection	Open
F-05	JWT Issuer/Audience Validation Gap Enables Role Confusion	High	8.1	A07: Authentication Failures	Open
F-06	CORS Origin Reflection with Credentialed API Access	High	8.2	A02: Security Misconfiguration	Open
F-07	Unauthenticated Debug Configuration Endpoint Exposure	High	7.5	A02: Security Misconfiguration	Open
F-08	Active Content Upload Served from Trusted Asset Domain	Medium	6.1	A08: Software/Data Integrity Failures	Open

Risk Rating Key

Rating	Remediation Priority
Critical	Immediate exploitation likely or severe business impact; target remediation within 7 calendar days.
High	High likelihood or material impact; target remediation within 15 calendar days.
Medium	Moderate exploitability or impact; target remediation within 30 calendar days.
Low	Limited impact or requiring unusual preconditions; target remediation in planned hardening cycles.

Chaining risk: Stored XSS, permissive CORS and weak authorization controls can amplify each other. Remediation should consider attack chains rather than treating each item as an isolated defect.

7. Detailed Vulnerability Observations and Proofs of Concept

Evidence standard: Each proof of concept was executed against agreed in-scope staging assets using approved test accounts. Request and response excerpts are sanitized but preserve the technical behavior necessary for remediation.

F-01	High	CVSS 8.1	Open
------	------	----------	------

Broken Object Level Authorization in Invoice Retrieval

Affected Component	GET /api/v2/tenants/{tenantId}/invoices/{invoiceId}
OWASP Mapping	OWASP Top 10 2025 A01 - Broken Access Control
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

Observation Summary

The invoice retrieval API accepted a valid invoice identifier from a different merchant tenant and returned invoice details even though the authenticated user belonged to another tenant. The UI restricted access, but the API did not perform equivalent object-level authorization.

Preconditions

Authenticated as qa.vendor.viewer@nexora.local with read-only access to tenant NTX-MER-021. Auditor used invoice identifiers discovered through predictable numbering during authorized testing.

Step	Proof-of-Concept Activity
1	Authenticate as Vendor Viewer for tenant NTX-MER-021.
2	Send a legitimate invoice request and capture the request in the proxy.
3	Replace only the invoiceId value with INV-2026-004918, belonging to NTX-MER-044.
4	Observe HTTP 200 response containing another tenant's invoice metadata and payout account suffix.

Broken Object Level Authorization - Invoice RetrievalEVD-01

RepeaterProxyLogger

REQUEST AS VENDOR VIEWER

GET /api/v2/tenants/NTX-MER-021/invoices/INV-2026-004918
HTTP/2
Host: api-stage.nexora-retail.example
Authorization: Bearer eyJ...vendor_viewer
X-Tenant-ID: NTX-MER-021
Accept: application/json

RESPONSE FROM OTHER TENANT

HTTP/2 200 OK
content-type: application/json
x-request-id: 49ee9f8c2a

{
 "invoiceId": "INV-2026-004918",
 "tenantId": "NTX-MER-044",
 "merchantName": "Orchid Foods Pvt. Ltd.",
 "payoutAccount": "****4418",
 "grossAmount": 482920.00,
 "customerEmail": "[redacted]@example.com"
}

Auditor noteAuthenticated as a low-privilege vendor viewer for NTX-MER-021, the auditor changed only the invoice identifier. The API returned invoice data for NTX-MER-044, confirming tenant authorization was not enforced server-side.

Evidence F-01 - Sanitized proof-of-concept capture

Business Impact

An attacker with any low-privilege merchant account could enumerate or access invoice records belonging to other merchants, exposing payment metadata, invoice totals and customer identifiers. Integrity impact exists where adjacent invoice actions reuse the same authorization pattern.

Likely Root Cause

Authorization logic trusts route parameters and UI restrictions instead of validating that the requested object belongs to the authenticated user's tenant on every API call.

Recommended Remediation

#	Action
1	Enforce object-level authorization in the service layer for every tenant-scoped resource.
2	Resolve the authenticated tenant from server-side claims and ignore client-supplied tenant identifiers for authorization decisions.
3	Use non-sequential object identifiers or add enumeration detection for sensitive resources.
4	Add negative authorization tests for cross-tenant object access to CI/CD.

Retest / Validation Criteria: Retest by replaying cross-tenant invoice IDs under all non-owner roles and verify HTTP 403/404 with no object metadata leakage.

Time-Based SQL Injection in Orders Search API

Affected Component	POST /api/v2/orders/search
OWASP Mapping	OWASP Top 10 2025 A05 - Injection
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H

Observation Summary

The order search API appears to concatenate the query field into a database statement. Controlled time-delay testing produced consistent database-side response delays, confirming injection without needing destructive or data-exfiltration payloads.

Preconditions

Authenticated merchant operator role. No elevated privileges were required beyond normal order search access.

Step	Proof-of-Concept Activity
1	Capture a normal order search request and record baseline response time.
2	Submit a controlled database time-delay payload in the query value.
3	Repeat the request five times across the testing window to rule out network variance.
4	Confirm the median delay increased from approximately 312 ms to approximately 5.86 seconds.

Time-Based SQL Injection - Orders Search API

EVD-02

RepeaterProxyLogger

INJECTED SEARCH REQUEST

```
POST /api/v2/orders/search HTTP/2
Host: api-stage.nexora-retail.example
Authorization: Bearer eyJ...merchant_operator
Content-Type: application/json

{
  "merchantId": "NTX-MER-021",
  "query": "laptop%' AND pg_sleep(5)--",
  "dateFrom": "2026-05-01",
  "page": 1
}
```

TIMING EVIDENCE

```
HTTP/2 200 OK
content-type: application/json
x-db-duration-ms: 5841
x-request-id: 2d2f15e1b7

{
  "total": 0,
  "elapsedMs": 5848,
  "items": []
}
```

Baseline query median: 312 ms
Injected query median across 5 attempts: 5.86 s

Auditor note

A controlled time-delay payload consistently increased response time by approximately five seconds. This indicates unsafely concatenated input in the order search query path. No destructive payloads were executed.

Evidence F-02 - Sanitized proof-of-concept capture

Business Impact

Successful exploitation could allow attackers to read or alter order data, bypass merchant-level filters, execute expensive database operations or pivot into application data stores depending on database permissions.

Likely Root Cause

Dynamic SQL construction is likely used for flexible search filters without parameterized queries or safe query builders.

Recommended Remediation

#	Action
1	Replace dynamic SQL concatenation with parameterized statements or vetted ORM query builders.

CONFIDENTIAL | Nexora Retail Services Pvt. Ltd. | Web Application VAPT Report | Page 13

#	Action
2	Implement allowlisted search fields and server-side query parsing.
3	Run database accounts with least privilege and separate read/write permissions.
4	Add SQL injection unit tests and SAST rules for query construction paths.
Retest / Validation Criteria: Confirm time-delay payloads no longer alter response time beyond normal variance and that database logs show prepared statement use for search queries.	

Password Reset OTP Endpoint Missing Rate Limiting

Affected Component	POST /auth/v1/password-reset/verify-otp
OWASP Mapping	OWASP Top 10 2025 A07 - Authentication Failures
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

Observation Summary

The password reset OTP verification endpoint accepted thousands of consecutive attempts for the authorized test account without lockout, throttling, step-up verification or risk challenge. A valid reset token was issued when the correct OTP was reached.

Preconditions

Knowledge of a registered user email and ability to trigger the password reset workflow. No authenticated session was required.

Step	Proof-of-Concept Activity
1	Initiate password reset for qa.vendor.operator@nexora.local.
2	Capture the OTP verification request in the proxy.
3	Send sequential OTP values under the agreed rate cap using the designated test account.
4	Observe no account lockout or rate-limit headers and a successful reset_token_issued response for the valid OTP.

Burp Intruder - OTP Verification Attack Window

EVD-03

Requests sent
3,000

Lockouts observed
0

Throttle headers
none

Success response
HTTP 200

#	OTP payload	Status	Length	Time	Result marker
2907	318407	401	312	184 ms	invalid_otp
2908	318408	401	312	181 ms	invalid_otp
2909	318409	401	312	183 ms	invalid_otp
2910	318410	401	312	186 ms	invalid_otp
2911	318411	401	312	179 ms	invalid_otp
2912	318412	401	312	187 ms	invalid_otp
2913	318413	200	948	196 ms	reset_token_issued
2914	318414	401	312	184 ms	invalid_otp
2915	318415	401	312	185 ms	invalid_otp

Auditor note

The password-reset OTP endpoint accepted continuous sequential attempts for the authorized test account and issued a reset token after the valid OTP was reached. No account lockout, IP throttling, or risk challenge was triggered.

Evidence F-03 - Sanitized proof-of-concept capture

Business Impact

An attacker could brute force OTPs for targeted accounts, obtain a password reset token and take over merchant or support accounts, depending on target role and monitoring effectiveness.

Likely Root Cause

OTP verification lacks per-account, per-IP and per-device attempt tracking. OTP entropy and validity window are not compensated by enforcement controls.

Recommended Remediation

#	Action
1	Add strict per-account and per-IP attempt limits with exponential backoff.

#	Action
2	Invalidate OTP after a small number of failed attempts and require restart of recovery flow.
3	Add risk-based step-up controls and alerting for repeated recovery attempts.
4	Log and monitor reset attempt patterns, including distributed attempts across IPs.
Retest / Validation Criteria: Retest sequential and distributed OTP attempts and verify lockout, telemetry and alert generation before reset token issuance.	

Stored Cross-Site Scripting in Support Ticket Comments

Affected Component	POST /api/v2/support/tickets/{ticketId}/comments
OWASP Mapping	OWASP Top 10 2025 A05 - Injection
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N

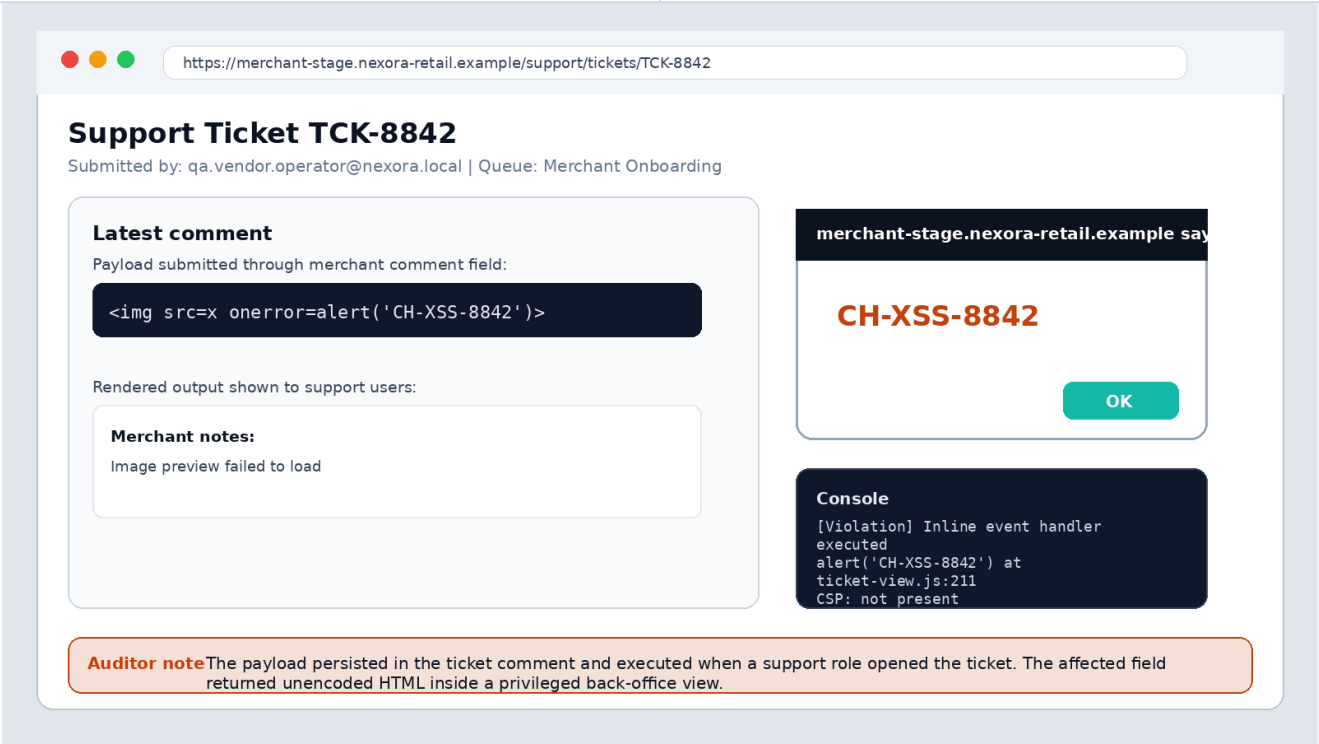
Observation Summary

HTML submitted by a merchant user in a support ticket comment was stored and later executed in the support agent portal when the ticket was opened. The issue affects a privileged back-office workflow.

Preconditions

Authenticated merchant user with permission to create or update support ticket comments. Victim support user must open the affected ticket.

Step	Proof-of-Concept Activity
1	Create a support ticket as Merchant Operator.
2	Submit a comment containing a benign JavaScript alert payload.
3	Log in as Support Agent and open the ticket in the back-office portal.
4	Observe JavaScript execution in the support user's browser context.



Evidence F-04 - Sanitized proof-of-concept capture

Business Impact

An attacker could execute arbitrary JavaScript in a support user's session, read sensitive ticket data, perform unauthorized actions through the victim's privileges or chain with weak CORS/session controls.

Likely Root Cause

User-generated comment content is rendered as HTML in the support console without contextual output encoding or sanitization. No effective Content Security Policy was present in the tested view.

Recommended Remediation

#	Action
1	Apply contextual output encoding for all user-controlled fields.
2	Sanitize rich-text input with a mature allowlist sanitizer if rich text is

#	Action
	required.
3	Deploy a restrictive Content Security Policy that blocks inline script execution.
4	Add regression tests for stored XSS in merchant-to-support workflows.
Retest / Validation Criteria: Verify stored payloads render as inert text and browser console shows CSP enforcement for inline script attempts.	

JWT Issuer/Audience Validation Gap Enables Role Confusion

Affected Component	API Gateway JWT validation middleware
OWASP Mapping	OWASP Top 10 2025 A07 - Authentication Failures
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

Observation Summary

The API gateway accepted a token issued for a staging support audience when accessing an administrative user endpoint. Authorization decisions used role claims without enforcing expected issuer and audience binding for the target API.

Preconditions

Possession of a valid token from another trusted Nexora audience or environment. The test used a sanctioned staging token with redacted values.

Step	Proof-of-Concept Activity
1	Decode a valid token issued by the staging identity provider for the support audience.
2	Replay the token against an API endpoint expecting CommerceHub portal audience.
3	Observe the API accepts the token and evaluates the role claim.
4	Confirm administrative user metadata is returned for the requested tenant.

JWT Validation Gap - Staging Token Accepted by Admin API

EVD-05

RepeaterProxyLogger

ADMIN REQUEST

GET /api/v2/admin/users?tenant=NTX-MER-021 HTTP/2
Host: api-stage.nexora-retail.example
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9. [redacted].sig
X-Client-App: CommerceHub-Web

API RESPONSE

HTTP/2 200
content-type: application/json

Decoded JWT payload

{
 "iss": "https://auth-staging.nexora-retail.example",
 "aud": "commercehub-support",
 "sub": "qa.vendor.viewer",
 "role": "support_admin"
}

{
 "requestedRole": "support_admin",
 "effectiveRole": "support_admin",
 "tokenIssuer": "auth-staging",
 "users": [
 {
 "email": "[redacted]@merchant.example",
 "role": "MerchantAdmin"
 }
]
}

Auditor note

A token issued for the staging support audience was accepted by the API gateway for an administrative endpoint. The API did not enforce expected issuer/audience binding before authorization decisions.

Evidence F-05 - Sanitized proof-of-concept capture

Business Impact

Role confusion can allow access across applications or environments where tokens share signing trust but differ in intended audience. Attackers could bypass intended separation between support, merchant and staging contexts.

Likely Root Cause

JWT validation checks token signature and expiry but does not strictly validate issuer, audience, authorized party and environment-specific trust boundaries before authorization.

Recommended Remediation

#	Action
1	Enforce exact issuer and audience validation per API service and route

#	Action
	group.
2	Use separate signing keys or key identifiers per environment and application audience.
3	Reject tokens containing roles not issued for the target resource server.
4	Add automated token validation tests for wrong issuer, wrong audience and environment-crossing tokens.
Retest / Validation Criteria: Replay tokens from non-target audiences and verify consistent HTTP 401/403 responses before business logic executes.	

CORS Origin Reflection with Credentialed API Access

Affected Component	API gateway CORS policy
OWASP Mapping	OWASP Top 10 2025 A02 - Security Misconfiguration
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N

Observation Summary

The API reflected arbitrary Origin values and set Access-Control-Allow-Credentials: true for authenticated endpoints. This permits malicious origins to read responses from victims with active sessions under browser enforcement rules.

Preconditions

Victim is authenticated to CommerceHub in the same browser. Attacker controls a website that can induce the victim to visit it.

Step	Proof-of-Concept Activity
1	Send an authenticated API request with an untrusted Origin header.
2	Observe the API reflects the untrusted origin in Access-Control-Allow-Origin.
3	Confirm Access-Control-Allow-Credentials: true is present.
4	Validate JSON profile data is returned in the response body.

CORS Origin Reflection with Credentialed API Access

EVD-06

RepeaterProxyLogger

CROSS-ORIGIN REQUEST

```
GET /api/v2/profile HTTP/2
Host: api-stage.nexora-retail.example
Origin: https://partner-tools.attacker.example
Cookie: NSID=s%3Aredacted; SameSite=None; Secure
Accept: application/json
```

CREDENTIALS ALLOWED

```
HTTP/2 200 OK
content-type: application/json
access-control-allow-origin:
https://partner-tools.attacker.example
access-control-allow-credentials: true
vary: Origin

{
  "userId": "USR-7721",
  "email": "qa.vendor.operator@nexora.local",
  "merchantId": "NTX-MER-021",
  "permissions": ["orders:read","refunds:create"]
}
```

Auditor note

The API reflected an untrusted Origin and enabled credentials. A browser session on an attacker-controlled origin could read JSON responses from authenticated endpoints where the victim has an active session.

Evidence F-06 - Sanitized proof-of-concept capture

Business Impact

An attacker-controlled website could read sensitive JSON responses from authenticated users, including profile metadata, permissions and potentially order or refund information depending on endpoint exposure.

Likely Root Cause

CORS middleware dynamically reflects Origin headers and enables credentialed access without allowlisting trusted origins per environment and route.

Recommended Remediation

#	Action
1	Replace origin reflection with an explicit allowlist of trusted HTTPS origins.
2	Disable credentialed CORS unless required for a specific trusted web

#	Action
	client.
3	Apply route-specific CORS policies and deny CORS by default for sensitive APIs.
4	Add negative CORS tests for untrusted origins to deployment gates.
Retest / Validation Criteria: Requests from untrusted origins should omit Access-Control-Allow-Origin and Access-Control-Allow-Credentials; browser fetch attempts should fail CORS checks.	

F-07	High	CVSS 7.5	Open
------	------	----------	------

Unauthenticated Debug Configuration Endpoint Exposure

Affected Component	GET /__debug/config
OWASP Mapping	OWASP Top 10 2025 A02 - Security Misconfiguration
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Observation Summary

A debug configuration endpoint was exposed without authentication and returned application configuration, internal hostnames, feature flags and secret material. Sensitive values were redacted in the evidence after validation.

Preconditions

Network access to the staging portal. No authenticated session was required.

Step	Proof-of-Concept Activity
1	Browse directly to /__debug/config on the staging portal host.
2	Observe HTTP 200 response with JSON configuration values.
3	Validate that returned values include internal database hostnames and credential-like secrets.
4	Capture sanitized evidence and cease further access to secret material.

Browser - Debug Configuration ExposureEVD-07

https://merchant-stage.nexora-retail.example/__debug/config

```
HTTP/2 200 OK
content-type: application/json
cache-control: no-store

{
  "app": "commercehub-portal",
  "spring.profiles.active": "stage,debug",
  "db.url": "jdbc:postgresql://pg-stage-01.internal:5432/commercehub",
  "db.user": "ch_stage_app",
  "jwt.signingKey": "[REDACTED-32-BYTE-SECRET]",
  "aws.accessKeyId": "AKIA*****Q4T",
  "sentry.dsn": "https://[redacted]@o999999.ingest.sentry.io/450",
  "featureFlags.adminBulkRefund": true
}

Trace header: X-CH-Request-Id: 9af2e87d-35ec-4e0b-a41b-3b6e3fbb0a01
```

Auditor note The endpoint exposed environment configuration and credential material to unauthenticated users. Values above were redacted by the audit team after validation.

Evidence F-07 - Sanitized proof-of-concept capture

Business Impact

Exposed configuration can assist attackers in credential theft, environment mapping, targeted exploitation and lateral movement. If secrets are valid in adjacent environments, impact can increase substantially.

Likely Root Cause

Debug endpoints or actuator-style configuration routes are enabled in a web-accessible environment without authentication, network restriction or response redaction.

Recommended Remediation

#	Action
1	Disable debug routes outside local development environments.
2	Restrict operational endpoints to authenticated administrative networks

#	Action
	only.
3	Redact secret values in all diagnostic responses and rotate exposed secrets.
4	Add deployment checks that fail builds when debug profiles are enabled in shared environments.
Retest / Validation Criteria: Confirm /__debug/config returns 404 or 403 externally, secrets are rotated, and debug profile flags are disabled in deployment manifests.	

F-08	Medium	CVSS 6.1	Open
------	--------	----------	------

Active Content Upload Served from Trusted Asset Domain

Affected Component	POST /api/v2/documents/upload and assets-stage delivery
OWASP Mapping	OWASP Top 10 2025 A08 - Software/Data Integrity Failures
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N

Observation Summary

The document upload function accepted HTML content and served it from a trusted Nexora asset subdomain as text/html. Users viewing the file would execute attacker-supplied script in a domain visually associated with Nexora.

Preconditions

Authenticated merchant user with document upload permissions. Victim must open the public asset URL or receive it through a trusted workflow.

Step	Proof-of-Concept Activity
1	Upload a file named invoice-preview.html using the merchant document upload API.
2	Observe the API returns a public asset URL and preserves contentType text/html.
3	Request the public URL and confirm Content-Type: text/html is returned.
4	Confirm X-Content-Type-Options and download disposition protections are not present.

Active Content Upload Served from Trusted Asset Domain

RepeaterProxyLogger

UPLOAD REQUEST

POST /api/v2/documents/upload HTTP/2
Host: api-stage.nexora-retail.example
Authorization: Bearer eyJ...merchant_admin
Content-Type: multipart/form-data; boundary=----ch

-----ch
Content-Disposition: form-data; name="file";
filename="invoice-preview.html"
Content-Type: text/html

<script>fetch('/api/v2/profile').then(r=>r.text()).then(console.log)</script>
-----ch--

UPLOAD RESPONSE

HTTP/2 201 Created
content-type: application/json

{
 "documentId": "DOC-2026-77192",
 "storageKey":
 "tenant/NTX-MER-021/docs/invoice-preview.html",
 "publicUrl":
 "https://assets-stage.nexora-retail.example/u/NTX-MER-021/invoice-preview.html",
 "contentType": "text/html"
}

Follow-up GET returned: Content-Type: text/html;
X-Content-Type-Options: not present

Auditor note

The upload API accepted an HTML file and returned a public URL on the trusted asset domain. The file was served as text/html without download disposition, enabling script execution in a domain associated with Nexora assets.

Evidence F-08 - Sanitized proof-of-concept capture

Business Impact

The issue can support phishing, token-adjacent attack chains, browser-based pivoting and brand abuse. If asset domain cookies or privileged integrations are later introduced, impact may increase.

Likely Root Cause

Upload validation checks file size and extension inconsistently and the asset delivery layer preserves user-controlled active content types.

Recommended Remediation

#	Action
1	Block active content types such as HTML, SVG with script, JavaScript and XML unless explicitly required.
2	Serve untrusted uploads from a segregated domain with no application cookies or trust relationship.
3	Force Content-Disposition: attachment and X-Content-Type-Options: nosniff for untrusted files.
4	Perform server-side MIME validation and normalize file extensions.
Retest / Validation Criteria: Attempt to upload HTML/SVG/JS files and verify rejection or forced safe download from an isolated domain.	

8. Remediation Roadmap and References

Prioritized Remediation Roadmap

Target Window	Findings	Actions
0-7 days	F-02, F-03	Patch SQL injection path; enforce OTP lockout/rate limits; rotate any secrets exposed through debug endpoint.
8-15 days	F-01, F-04, F-05, F-06, F-07	Implement server-side authorization checks, output encoding/CSP, strict JWT validation, CORS allowlisting and remove debug endpoints.
16-30 days	F-08	Harden upload validation and segregate untrusted content delivery.
Ongoing	All	Add regression tests, centralized authorization libraries, secure coding gates and security telemetry dashboards.

Retesting Checklist

Control Area	Retest Expectation
Authorization	Cross-tenant and cross-role negative tests return 403/404 with no object metadata.
Injection	Parameterized queries confirmed; timing and error-based payloads fail safely.
Authentication	OTP rate limits, account lockouts and monitoring alerts confirmed under sequential and distributed attempts.
Browser security	Stored payloads render inert; CSP blocks inline script; CORS denies untrusted origins.
Configuration	Debug endpoints disabled; exposed secrets rotated; deployment checks prevent recurrence.
Uploads	Active content rejected or forced as safe download from isolated domain.

References

Reference	URL
OWASP Top Ten Web Application Security Risks	https://owasp.org/www-project-top-ten/
OWASP Web Security Testing Guide	https://owasp.org/www-project-web-security-testing-guide/
OWASP Application Security Verification Standard	https://github.com/OWASP/ASVS
FIRST CVSS v3.1 Specification Document	https://www.first.org/cvss/v3.1/specification-document

Closure note: Retesting should be performed after code remediation, configuration deployment and secret rotation are complete. Evidence from retesting should be appended to this report or tracked in Nexora's vulnerability management workflow.